

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Master Digitale Transformation

Modul Digitale Technologien

Einführung in Python

Grundlagen, Algorithmen und Datenanalyse

Prof. Dr.-Ing. Sebastian Schlesinger

Lernweg

1. Python: Geschichte, Eigenschaften und Einsatzfelder
2. Arbeitsumgebung: Colab, Python, VS Code, venv und Docker
3. Syntax, Funktionen und grundlegende Datenstrukturen
4. Algorithmen selbst programmieren und Aufwand verstehen
5. Klassen, Vererbung und verschiedene Programmierstile
6. Bibliotheken importieren, Daten mit pandas und Matplotlib auswerten

Wie Python entstand

Guido van Rossum begann die Entwicklung 1989 am CWI

1991 erschien die erste öffentliche Version

Einflüsse: ABC, lesbare Syntax und erweiterbare Systemwerkzeuge

Der Name bezieht sich auf Monty Python’s Flying Circus

Im Kurs verwenden wir Python 3

Einordnung der Sprache

Universelle höhere Programmiersprache mit dynamischer Typisierung

Unterstützt prozedurale, objektorientierte und funktionale Ansätze

Automatische Speicherverwaltung und umfangreiche Standardbibliothek

CPython übersetzt Quelltext in Bytecode und führt diesen aus

Typische Felder: Automatisierung, Webanwendungen und Datenanalyse

Stärken und Grenzen

Lesbarer Code unterstützt den Einstieg und die Zusammenarbeit

Viele Bibliotheken verbinden Python mit spezialisierten Werkzeugen

Reine Python-Schleifen können bei großen Datenmengen teuer sein

Typfehler können erst bei der Ausführung sichtbar werden

Die passende Lösung hängt von Aufgabe, Datenmenge und Umgebung ab

Welche Arbeitsumgebung übernimmt was?

Python-Interpreter: führt Python-Code aus

Colab: Notebook und Rechenumgebung im Browser

VS Code: Editor mit Erweiterungen für Python und Notebooks

venv: eigene Python-Pakete pro lokalem Projekt

Docker: paketiert zusätzlich die Laufzeit und Systemabhängigkeiten

Colab und Jupyter

colab.research.google.com öffnen und ein Python-Notebook anlegen

Alternativ: vorhandene .ipynb-Datei über das Dateimenü hochladen

Codezelle mit print("Hallo Python") anlegen und ausführen

Shift + Enter führt eine Zelle aus. Eine CPU-Laufzeit genügt.

Notebook herunterladen, bevor die Sitzung beendet wird

Notebook-Zustand und Ausführungsreihenfolge

Codezellen teilen sich den Zustand eines laufenden Kernels

Ein alter Variablenwert kann auch nach dem Ändern einer Zelle bleiben

Ein frischer Kernel beginnt ohne die bisherigen Variablen

Kontrolle: neu starten und alle Zellen von oben nach unten ausführen

Textzellen dokumentieren Frage, Vorgehen und Interpretation

Python lokal installieren

Python 3 von python.org installieren, im Kurs z. B. Python 3.12+

Version im Terminal prüfen: macOS/Linux mit python3 --version

Windows: py -3 --version oder python --version

Projektordner anlegen und eine Datei hallo.py darin speichern

```
print("Hallo Python")
name = "Ada"
print(f"Willkommen, {name}!")
```

VS Code für Python

VS Code installieren und den Projektordner öffnen

Microsoft-Erweiterung „Python“ installieren

Für .ipynb-Dateien zusätzlich die Erweiterung „Jupyter“ installieren

„Python: Select Interpreter“ öffnet die Interpreterauswahl

hallo.py ausführen und die Ausgabe im Terminal kontrollieren

venv: eine Umgebung pro Projekt

Im Projektordner eine virtuelle Umgebung erzeugen

macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
python --version
python hallo.py
```

venv unter Windows

PowerShell: Umgebung erstellen und direkt deren Python verwenden

Das funktioniert auch ohne Aktivierungsskript

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe hallo.py
```

Interpreter und Notebook-Kernel prüfen

In VS Code den Interpreter aus .venv auswählen

Bei Notebooks zusätzlich denselben Python-Kernel wählen

Das folgende Beispiel zeigt die tatsächlich verwendete Umgebung

```
import sys
print(sys.executable)
print(sys.version)
```

Docker: Image und Container

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Dockerfile beschreibt den Aufbau einer Laufzeitumgebung

Image ist die gebaute Vorlage. Container ist eine laufende Instanz.

Ein Container nutzt den Kernel seiner Laufzeitumgebung

Docker Desktop stellt für Linux-Container auf Mac/Windows eine VM bereit

venv isoliert Python-Pakete, Docker zusätzlich Systemabhängigkeiten

Docker: ein kleines Python-Skript

Optional: Docker installieren und starten

hallo.py und Dockerfile liegen im selben Ordner

```
FROM python:3.12-slim
WORKDIR /app
RUN useradd --create-home student
COPY hallo.py .
USER student
CMD ["python", "hallo.py"]
```

Docker bauen und starten

Die Befehle laufen im Terminal des Projektordners

build erzeugt das Image. run startet einen Container daraus.

```
docker build -t python-intro .
docker run --rm python-intro
```

Programme, Ausdrücke und Anweisungen

Ein Programm beschreibt Schritte, die der Rechner ausführt

Ausdrücke berechnen Werte. Anweisungen steuern den Ablauf.

Einrückung kennzeichnet zusammengehörende Codeblöcke

```
preis = 12.5
menge = 4
umsatz = preis * menge
print(f"Umsatz: {umsatz:.2f} Euro")
```

Variablen und Typen

Namen verweisen auf Objekte. Objekte besitzen einen Typ.

int: ganze Zahl, float: Gleitkommazahl, str: Text

bool: True/False, None: kein Wert

```
anzahl = 3
preis = 12.5
produkt = "Sensor"
verfuegbar = True
rabatt = None
print(type(preis))
```

Operatoren und Umwandlungen

= weist zu, == vergleicht Werte

/ dividiert, // rundet den Quotienten ab, % liefert den Rest

and, or und not verknüpfen Wahrheitswerte

```
preis = float("12.50")
print(7 / 2, 7 // 2, 7 % 2)
print(preis >= 10 and preis < 20)
print(f"Preis: {preis:.2f}")
```

Verzweigungen

Bedingungen entscheiden, welcher Block ausgeführt wird

Doppelpunkt und vier Leerzeichen strukturieren den Code

```
umsatz = 120
if umsatz >= 100:
    kategorie = "hoch"
elif umsatz >= 50:
    kategorie = "mittel"
else:
    kategorie = "niedrig"
print(kategorie)
```

Schleifen mit for

for durchläuft die Elemente einer Folge

range(3) liefert 0, 1 und 2. Das Ende gehört nicht dazu.

```
preise = [10, 20, 15]
gesamt = 0
for preis in preise:
    gesamt += preis
print(gesamt)  # 45
for index, preis in enumerate(preise):
    print(index, preis)
```

Schleifen mit while

while wiederholt, solange eine Bedingung wahr ist

Der Schleifenrumpf muss die Abbruchbedingung erreichbar machen

```
bestand = 3
while bestand > 0:
    print("Verkauf")
    bestand -= 1
print("Ausverkauft")
```

Listen als erster Datenspeicher

Eine Liste speichert eine geordnete, veränderbare Folge

Index 0 bezeichnet das erste Element. -1 das letzte.

Slicing: Start eingeschlossen, Ende ausgeschlossen

```
werte = [12, 8, 15]
werte.append(20)
print(werte[0], werte[-1])
print(werte[1:3])  # [8, 15]
print([x * 2 for x in werte if x >= 12])
```

Übung: Umsätze selbst berechnen

Preise: [12, 8, 15, 10], Mengen: [2, 5, 1, 3]

Mit einer Schleife Einzelumsätze und Gesamtumsatz berechnen

Zählen, wie viele Umsätze mindestens 30 Euro betragen

Erwartung: 109 Euro insgesamt und zwei passende Umsätze

Funktionen

Eine Funktion bündelt eine wiederverwendbare Berechnung

Parameter liefern Eingaben. return liefert das Ergebnis.

```
def netto(preis, rabatt=0.0):
    return preis * (1 - rabatt)

betrag = netto(100, rabatt=0.1)
print(betrag)  # 90.0
```

Verträge, Fehler und kleine Tests

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Typannotationen dokumentieren erwartete Typen

Validierung prüft fachliche Voraussetzungen zur Laufzeit

Tests prüfen Beispiele und Randfälle

```
def netto(preis: float, rabatt: float = 0.0) -> float:
    if preis < 0 or not 0 <= rabatt <= 1:
        raise ValueError("Preis oder Rabatt unzulässig")
    return preis * (1 - rabatt)

assert netto(100, 0.1) == 90
assert netto(100, 1.0) == 0
```

Fehler lesen und gezielt behandeln

SyntaxError: Python kann den Code nicht lesen

NameError: Ein Name ist unbekannt

TypeError / ValueError: Typ oder Wert passt nicht

```
eingabe = "zwölf"
try:
    preis = float(eingabe)
except ValueError:
    print("Bitte eine Zahl wie 12.5 eingeben")
```

Datenstrukturen im Überblick

list: veränderbare Folge, etwa Messwerte in zeitlicher Reihenfolge

tuple: unveränderbare Folge, etwa ein Koordinatenpaar

dict: Zuordnung von Schlüsseln zu Werten, etwa Produktpreise

set: eindeutige Elemente, etwa alle vorkommenden Kategorien

Referenzen und Kopien

Zuweisung kopiert bei Listen nicht deren Inhalt

Eine flache Kopie erzeugt eine neue äußere Liste

```
a = [1, 2]
b = a
b.append(3)
print(a)  # [1, 2, 3]
c = a.copy()
c.append(4)
print(a)  # [1, 2, 3]
```

Hashmaps mit dict und Mengen mit set



dict arbeitet in Python als Hashtabelle

Ein Schlüssel kommt höchstens einmal vor

dict erhält die Einfügereihenfolge. set hat keine garantierte Reihenfolge.

```
preise = {"Sensor": 12.5, "Kabel": 3.0}
preise["Adapter"] = 7.0
preise["Sensor"] = 13.0
print(preise.get("Motor", 0))
print(set(["A", "B", "A"]))
```

Stack und Queue mit Listen

Stack: zuletzt hinein, zuerst heraus. Queue: zuerst hinein, zuerst heraus.

Wir verwenden zunächst nur Listenoperationen

```
stack = []
stack.append("A")
stack.append("B")
print(stack.pop())    # B
queue = ["A", "B"]
queue.append("C")
print(queue.pop(0))   # A
```

Aufwand von Operationen

O(1): Aufwand wächst nicht mit der Anzahl n der Elemente

O(log n): Suchraum schrumpft wiederholt um einen Faktor

O(n): Aufwand wächst ungefähr proportional zu n

O(n log n): typisch für effizientes vergleichsbasiertes Sortieren

O(n^2): doppelte Durchläufe über wachsende Datenmengen

Lineare Suche

Beliebige Reihenfolge möglich

Bis zu n Vergleiche. Laufzeit O(n).

```
def linear_suche(werte, gesucht):
    for i, wert in enumerate(werte):
        if wert == gesucht:
            return i
    return -1

print(linear_suche([7, 2, 9], 2))  # 1
```

Binäre Suche: die Idee

Voraussetzung: eine aufsteigend sortierte Liste

Linke und rechte Grenze beschreiben den verbleibenden Suchbereich

Vergleich mit der Mitte verwirft jeweils eine Hälfte

Beispiel: 11 in [2, 5, 8, 11, 17] suchen

Laufzeit O(log n), Sortieren ist ein zusätzlicher Vorbereitungsschritt

Binäre Suche selbst implementiert

-1 bedeutet hier „nicht gefunden“. Bei Duplikaten genügt ein Treffer.

```
def binaer_suche(a, ziel):
    links, rechts = 0, len(a) - 1
    while links <= rechts:
        mitte = (links + rechts) // 2
        if a[mitte] == ziel:
            return mitte
        if a[mitte] < ziel:
            links = mitte + 1
        else:
            rechts = mitte - 1
    return -1
```

Einfügen in eine sortierte Liste

Zunächst die Position suchen, dann größere Elemente verschieben

```
def einfuegen(a, wert):
    a.append(wert)
    i = len(a) - 2
    while i >= 0 and a[i] > wert:
        a[i + 1] = a[i]
        i -= 1
    a[i + 1] = wert

zahlen = [2, 5, 8]
einfuegen(zahlen, 6)
print(zahlen)  # [2, 5, 6, 8]
```

Insertion Sort als Lernalgorithmus

Jedes neue Element wandert an seinen Platz im sortierten Präfix

Beispiel: [5, 2, 4] wird [2, 5, 4], dann [2, 4, 5]

```
def insertion_sort(werte):
    a = list(werte)
    for i in range(1, len(a)):
        x, j = a[i], i - 1
        while j >= 0 and a[j] > x:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = x
    return a
```

Algorithmen mit Randfällen prüfen

Tests prüfen Verhalten bei leerer Eingabe, Duplikaten und fehlenden Werten

```
assert linear_suche([], 3) == -1
assert binaer_suche([2, 5, 8], 8) == 2
assert binaer_suche([2, 5, 8], 4) == -1
assert insertion_sort([5, 2, 4]) == [2, 4, 5]
assert insertion_sort([3, 3, -1]) == [-1, 3, 3]
a = []
einfuegen(a, 4)
assert a == [4]
```

Klassen und Objekte

Klasse: Bauplan für Zustand und Verhalten

Objekt: konkrete Instanz einer Klasse

Attribute speichern Zustand. Methoden beschreiben Verhalten.

```
class Produkt:
    def __init__(self, name, preis):
        self.name = name
        self.preis = preis

    def umsatz(self, menge):
        return self.preis * menge
```

Objekte verwenden

Jede Instanz besitzt ihren eigenen Zustand

Kapselung bündelt zusammengehörige Daten und Operationen

```
sensor = Produkt("Sensor", 12.5)
kabel = Produkt("Kabel", 3.0)
print(sensor.name)
print(sensor.umsatz(4))   # 50.0
print(kabel.umsatz(4))    # 12.0
```

Vererbung und Überschreiben

Digitalprodukt übernimmt Attribute und Methoden von Produkt

Eine eigene Methode kann das Verhalten der Basisklasse überschreiben

```
class Digitalprodukt(Produkt):
    def __init__(self, name, preis, format):
        super().__init__(name, preis)
        self.format = format

    def umsatz(self, menge):
        return super().umsatz(menge) * 0.9
```

Polymorphie: gleicher Aufruf, anderes Verhalten



Die Schleife verwendet dieselbe Methode für unterschiedliche Objekte

```
produkte = [Produkt("Sensor", 10),
            Digitalprodukt("E-Book", 10, "PDF")]
for produkt in produkte:
    print(produkt.umsatz(2))
# 20
# 18.0
```

Duck Typing ohne gemeinsame Basisklasse



Auch unabhängig definierte Klassen können dieselbe Schnittstelle anbieten

```
class Pauschale:
    def umsatz(self, menge):
        return 50

for objekt in [Produkt("Sensor", 10), Pauschale()]:
    print(objekt.umsatz(2))  # 20, dann 50
```

Komposition: Objekte zusammensetzen



Ein Warenkorb enthält Produkte. Er ist selbst kein Produkt.

```
class Warenkorb:
    def __init__(self):
        self.produkte = []

    def hinzufuegen(self, produkt):
        self.produkte.append(produkt)

    def gesamt(self):
        betrag = 0
        for p in self.produkte:
            betrag += p.umsatz(1)
        return betrag
```

Binäre Suchbäume

Jeder Knoten speichert einen Wert und bis zu zwei Kinder

Links liegen kleinere, rechts größere Werte

Beispiel: Wurzel 8, Kinder 3 und 10, unter 3 die Werte 1 und 6

Suche nach 6 besucht 8, dann 3, dann 6

Suche und Einfügen kosten O(h), abhängig von der Baumhöhe h

Suchbaum: Knoten als Klasse



Jeder Knoten enthält einen Wert und Verweise auf seine Kinder

```
class Knoten:
    def __init__(self, wert):
        self.wert = wert
        self.links = None
        self.rechts = None

wurzel = Knoten(8)
wurzel.links = Knoten(3)
wurzel.rechts = Knoten(10)
```

Suche im Baum ohne Bibliothek

Die Suche folgt pro Schritt genau einem Kind

```
def baum_suche(knoten, ziel):
    while knoten is not None:
        if ziel == knoten.wert:
            return True
        if ziel < knoten.wert:
            knoten = knoten.links
        else:
            knoten = knoten.rechts
    return False
```

Programmierstile und Ablauftechniken



Prozedural: Aufgaben durch Anweisungen und Funktionen zerlegen

Objektorientiert: Zustand und Verhalten in Objekten bündeln

Funktional: Werte durch möglichst nebenwirkungsfreie Funktionen abbilden

Iteration: Schritte wiederholen. Rekursion: Funktion ruft sich selbst auf.

Iteration und Rekursion lassen sich mit mehreren Stilen kombinieren

Prozedural: Summe schrittweise berechnen

Eine lokale Variable sammelt die Zwischenergebnisse

```
def quadratsumme(werte):
    gesamt = 0
    for x in werte:
        gesamt += x * x
    return gesamt

print(quadratsumme([1, 2, 3]))  # 14
```

Funktional: Funktionen und Abbildungen

Funktionen können als Werte übergeben werden

Eine reine Funktion verändert keinen äußeren Zustand

```
def quadrat(x):
    return x * x

def anwenden(funktion, werte):
    return [funktion(x) for x in werte]

print(anwenden(quadrat, [1, 2, 3]))  # [1, 4, 9]
print(anwenden(lambda x: x + 1, [1, 2, 3]))
```

Iteration: Fakultät mit einer Schleife



Für ganze n ≥ 0 gilt n! = 1 · 2 · … · n und 0! = 1

```
def fakultaet_iterativ(n):
    if n < 0:
        raise ValueError("n muss nichtnegativ sein")
    ergebnis = 1
    for faktor in range(2, n + 1):
        ergebnis *= faktor
    return ergebnis

print(fakultaet_iterativ(4))  # 24
```

Rekursion: Fakultät durch Selbstaufruf



Basisfall beendet die Rekursion. Jeder Aufruf nähert sich ihm.

```
def fakultaet_rekursiv(n):
    if n < 0:
        raise ValueError("n muss nichtnegativ sein")
    if n <= 1:
        return 1
    return n * fakultaet_rekursiv(n - 1)

print(fakultaet_rekursiv(4))  # 24
```

Iteration und Rekursion im Vergleich

Beide Fakultätsvarianten führen O(n) Multiplikationen aus

Iteration speichert den Fortschritt in lokalen Variablen

Rekursion benötigt hier O(n) zusätzliche Aufrufrahmen

Rekursion passt oft zu verschachtelten Strukturen wie Bäumen

Für lange lineare Abläufe ist eine Schleife häufig einfacher

Rekursiv in einen Suchbaum einfügen

Die Funktion liefert die Wurzel des aktualisierten Teilbaums zurück

```
def baum_einfuegen(knoten, wert):
    if knoten is None:
        return Knoten(wert)
    if wert < knoten.wert:
        knoten.links = baum_einfuegen(knoten.links, wert)
    elif wert > knoten.wert:
        knoten.rechts = baum_einfuegen(knoten.rechts, wert)
    return knoten
```

Baum in sortierter Reihenfolge durchlaufen



Inorder: erst links, dann der Knoten, dann rechts

```
def inorder(knoten):
    if knoten is None:
        return
    inorder(knoten.links)
    print(knoten.wert)
    inorder(knoten.rechts)

wurzel = baum_einfuegen(wurzel, 6)
inorder(wurzel)  # 3, 6, 8, 10 in getrennten Zeilen
```

Übung D: passende Datenstruktur



Produktpreis anhand einer Artikelnummer finden: welche Struktur?

Die letzte Bearbeitung rückgängig machen: welche Struktur?

Viele Werte einmal suchen oder wiederholt suchen: Unterschied?

Zusatz: Suchbaum mit 1, 2, 3, 4, 5 befüllen. Was passiert?

Übung: Stile und Objekte vergleichen

Fakultät iterativ und rekursiv für 0, 1 und 5 prüfen

Digitalprodukt und Produkt im selben Warenkorb verwenden

Einen Baum mit 1, 2, 3, 4, 5 aufbauen und nach 5 suchen

Erklären: Warum braucht Polymorphie keine if-Abfrage nach dem Typ?

Module und Bibliotheken importieren

Ein Modul bündelt Python-Code, etwa in einer .py-Datei

import bindet ein Modul ein. from importiert einzelne Namen.

Ein Alias verkürzt den Namen, ohne die Funktionalität zu ändern

```
import math
from math import sqrt
import math as m
print(math.sqrt(9), sqrt(9), m.sqrt(9))
```

Zusätzliche Pakete installieren

Im Terminal der aktivierten Projektumgebung:

```
python -m pip install pandas matplotlib
python -m pip freeze > requirements.txt
```

pandas und Matplotlib

pandas: Tabellen mit benannten Spalten und Datenoperationen

Matplotlib: Diagramme und beschriftete Achsen

NumPy: numerische Arrays, häufig Grundlage anderer Bibliotheken

Zuerst haben wir die Abläufe selbst implementiert

Jetzt verwenden wir Bibliotheken für wiederkehrende Analyseaufgaben

pandas: eine Tabelle erzeugen

Eine Zeile beschreibt hier einen Verkauf

```
import pandas as pd
verkauf = pd.DataFrame({
    "produkt": ["Sensor", "Kabel", "Sensor"],
    "preis": [12.5, 3.0, 12.5],
    "menge": [2, 4, 1]
})
verkauf["umsatz"] = verkauf["preis"] * verkauf["menge"]
print(verkauf)
```

pandas: filtern und gruppieren

Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Filter wählen Zeilen aus. Gruppierung fasst ähnliche Zeilen zusammen.

```
print(verkauf[verkauf["umsatz"] >= 20])
umsaetze = verkauf.groupby("produkt")["umsatz"].sum()
print(umsaetze)
# Kabel     12.0
# Sensor    37.5
```

CSV und Datenqualität

CSV-Dateien enthalten tabellarische Daten als Text

Trennzeichen, Dezimalformat und Spaltentypen prüfen

```
verkauf.to_csv("verkauf.csv", index=False)
gelesen = pd.read_csv("verkauf.csv")
print(gelesen.head())
gelesen.info()
print(gelesen.isna().sum())
```

Matplotlib: Umsatz nach Produkt

Die gruppierten Werte werden als Balkendiagramm dargestellt

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.bar(umsaetze.index, umsaetze.values)
ax.set(xlabel="Produkt", ylabel="Umsatz (Euro)",
       title="Umsatz unserer Beispielverkäufe")
fig.tight_layout()
plt.show()
```

Übung: eigene kleine Datenanalyse

Eine zusätzliche Verkaufszeile für einen Adapter ergänzen

Umsatz pro Zeile und Gesamtumsatz je Produkt berechnen

Ein beschriftetes Balkendiagramm erzeugen

Zwei Sätze zur Aussage und zu den Grenzen der Daten schreiben

Key Takeaways

Python unterstützt mehrere Programmierstile in derselben Sprache

Eine passende Umgebung macht Ausführung und Abhängigkeiten nachvollziehbar

Eigene Implementierungen zeigen, wie Suche und Sortierung arbeiten

Klassen, Vererbung und Polymorphie strukturieren Verhalten

pandas und Matplotlib vereinfachen Tabellenarbeit und Visualisierung

Dokumentation und weitere Schritte



Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Python: docs.python.org/3/tutorial

Setup: code.visualstudio.com/docs/python/python-tutorial

Umgebungen: docs.python.org/3/library/venv.html

Tabellen: pandas.pydata.org/docs/getting_started

Grafiken: matplotlib.org/stable/users/explain/quick_start.html